

assignmentB

DSA Assignment #B : 20260513模拟考

Updated 2026-05-19 21:15 GMT+8

Compiled by 徐前 物理学院 (2026 Spring)

说明:

1. 解题与记录:

对于每一个题目，请提供其解题思路（可选），并附上使用Python或C++编写的源代码（确保已在OpenJudge, Codeforces, LeetCode等平台上获得Accepted）。请将这些信息连同显示“Accepted”的截图一起填写到下方的作业模板中。（推荐使用Typora <https://typoraio.cn> 进行编辑，当然你也可以选择Word。）无论题目是否已通过，请标明每个题目大致花费的时间。

2. **提交安排**：提交时，请首先上传PDF格式的文件，并将.md或.doc格式的文件作为附件上传至右侧的“作业评论”区。确保你的Canvas账户有一个清晰可见的本人头像，提交的文件为PDF格式，并且“作业评论”区包含上传的.md或.doc附件。

3. **延迟提交**：如果你预计无法在截止日期前提交作业，请提前告知具体原因。这有助于我们了解情况并可能为你提供适当的延期或其他帮助。

请按照上述指导认真准备和提交作业，以保证顺利完成课程要求。

1. 题目

E02724: 生日相同

sortings, <http://cs101.openjudge.cn/pctbook/E02724/>

思路:

注意判断同生日的人数**大于1**!!!

代码:

```
from collections import defaultdict
def main():
    n = int(input())
    date_id = defaultdict(list)
    for _ in range(n):
        line = input().split()
        date_id[(int(line[1]), int(line[2]))].append(line[0])
```

```

for key in sorted(date_id.keys()):
    if len(date_id[key]) > 1:
        print(*key, sep=' ', end=' ')
        print(*date_id[key], sep=' ')

if __name__ == '__main__':
    main()

```

代码运行截图 (至少包含有"Accepted")

#52581212提交状态

查看 提交 统计 提问

状态: Accepted

源代码

```

from collections import defaultdict
def main():
    n = int(input())
    date_id = defaultdict(list)
    for _ in range(n):
        line = input().split()
        date_id[(int(line[1]), int(line[2]))].append(line[0])
    for key in sorted(date_id.keys()):
        if len(date_id[key]) > 1:
            print(*key, sep=' ', end=' ')
            print(*date_id[key], sep=' ')

if __name__ == '__main__':
    main()

```

基本信息

#: 52581212
 题目: E02724
 提交人: 25n2500011619(xq)
 内存: 3612kB
 时间: 23ms
 语言: Python3
 提交时间: 2026-05-19 11:33:55

E19963: 买学区房

math, <http://cs101.openjudge.cn/practice/19963>

思路:

排序找中位数, 按题意实现。

代码:

```

def main():
    n = int(input())
    pairs = [i[1:-1] for i in input().split()]
    distances = [sum(map(int, i.split(','))) for i in pairs]
    costs = list(map(int, input().strip().split()))
    equality = [distances[i] / costs[i] for i in range(n)]
    sorted_costs = sorted(costs)
    sorted_equality = sorted(equality)

    if n & 1:
        mid_cost = sorted_costs[n // 2]
        mid_equality = sorted_equality[n // 2]

```

```

else:
    mid_cost = (sorted_costs[n // 2] + sorted_costs[n // 2 - 1]) / 2
    mid_equality = (sorted_equality[n // 2] + sorted_equality[n // 2 - 1])
/ 2
ans = 0
for i in range(n):
    if costs[i] < mid_cost and equality[i] > mid_equality:
        ans += 1
print(ans)

if __name__ == '__main__':
    main()

```

代码运行截图（至少包含有"Accepted"）

#52581225提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: **Accepted**

源代码

```

def main():
    n = int(input())
    pairs = [i[1:-1] for i in input().split()]
    distances = [sum(map(int, i.split(','))) for i in pairs]
    costs = list(map(int, input().strip().split()))
    equality = [distances[i] / costs[i] for i in range(n)]
    sorted_costs = sorted(costs)
    sorted_equality = sorted(equality)

    if n & 1:
        mid_cost = sorted_costs[n // 2]
        mid_equality = sorted_equality[n // 2]
    else:
        mid_cost = (sorted_costs[n // 2] + sorted_costs[n // 2 - 1]) / 2
        mid_equality = (sorted_equality[n // 2] + sorted_equality[n // 2 - 1]) / 2
    ans = 0
    for i in range(n):
        if costs[i] < mid_cost and equality[i] > mid_equality:
            ans += 1

```

基本信息

#: 52581225
 题目: 19963
 提交人: 25n2500011619(xq)
 内存: 4336kB
 时间: 26ms
 语言: Python3
 提交时间: 2026-05-19 11:35:25

M20746: 满足合法工时的最少人数

binary search, <http://cs101.openjudge.cn/practice/20746/>

思路:

二分+判断, 注意是求员工人数的最小值。

代码:

```

def check(k):
    tot = 0
    for x in a:
        tot += (x + k - 1) // k
    return tot <= t

```

```

a = list(map(int, input().split(' ')))
t = int(input())

l = max(sum(a) // t - 1, 0)
r = max(a)
while l < r - 1:
    mid = (l + r) // 2
    if check(mid):
        r = mid
    else:
        l = mid
print(r)

```

代码运行截图（至少包含有"Accepted"）

#52581237提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: **Accepted**

源代码

```

def check(k):
    tot = 0
    for x in a:
        tot += (x + k - 1) // k
    return tot <= t

a = list(map(int, input().split(' ')))
t = int(input())

l = max(sum(a) // t - 1, 0)
r = max(a)
while l < r - 1:
    mid = (l + r) // 2
    if check(mid):
        r = mid
    else:
        l = mid
print(r)

```

基本信息

#: 52581237
 题目: 20746
 提交人: 25n2500011619(xq)
 内存: 4764kB
 时间: 36ms
 语言: Python3
 提交时间: 2026-05-19 11:36:52

M07734: 虫子的生活

DSU, <http://cs101.openjudge.cn/practice/07734/>

思路:

用邻接表又不超内存，必须强制 Python 复用整数对象。可以预先创建一个包含 1 到 N 的整数列表，每次读取边时，用预设的对象替换掉新生成的对象。

优化点：在 `adj[u].append(nodes[v])` 时，存储的是这2000 个对象的引用，而不是每次 `input` 产生的新的整数对象。

代码:

```
import sys
```

```

input = sys.stdin.readline

def solve(case):
    line = input().split()
    while not line:
        line = input().split()
        if not line: return

    n, m = map(int, line)

    edge = [[] for _ in range(n + 1)]
    nodes = list(range(n + 1))

    for _ in range(m):
        a, b = map(int, input().split())
        edge[a].append(nodes[b])
        edge[b].append(nodes[a])

    has_odd_cycle = False
    color = bytearray(n + 1) # 0 表示未访问, 1 和 2 分别表示两种颜色

    for start in range(1, n + 1):
        if has_odd_cycle:
            break

        if color[start] == 0:
            # 迭代栈, 元素为 (节点, 颜色)
            stack = [start]
            color[start] = 1
            while stack:
                u = stack.pop()
                for v in edge[u]:
                    if color[v] == 0:
                        color[v] = 3 - color[u] # 1->2, 2->1
                        stack.append(v)
                    elif color[v] == color[u]:
                        has_odd_cycle = True
            if has_odd_cycle:
                break

    sys.stdout.write(f'Scenario #{case}:\n')
    if has_odd_cycle:
        sys.stdout.write('Suspicious bugs found!\n\n')
    else:
        sys.stdout.write('No suspicious bugs found!\n\n')

```

```
def main():
    line = input().strip()
    if line:
        for i in range(1, int(line) + 1):
            solve(i)

if __name__ == '__main__':
    main()
```

代码运行截图 (至少包含有"Accepted")

#52581261提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: Accepted

源代码

```
import sys
# import array
color = [0 for _ in range(2005)]
def check(n, edge):
    color[:n] = [0] * n # 0 表示未访问, 1 和 2 分别表示两种颜色

    for start in range(1, n):
        if color[start] == 0:
            stack = [start]
            color[start] = 1
            while stack:
                u = stack.pop()
                for v in range(1, n):
                    if edge[u][v] == 0: continue
                    if color[v] == 0:
                        color[v] = 3 - color[u] # 1->2, 2->1
                        stack.append(v)
```

基本信息

#: 52581261
 题目: 07734
 提交人: 25n2500011619(xq)
 内存: 106496kB
 时间: 1756ms
 语言: PyPy3
 提交时间: 2026-05-19 11:39:07

M02186: Popular Cows

SCC, <http://cs101.openjudge.cn/practice/02186/>

思路:

用 Tarjan 方法进行强连通分量缩点, 然后在有向无环图 (DAG) 上找到出度为 0 的点, 如果点数大于 1, 则不存在最受欢迎的奶牛, 否则输出这个 SCC 的联通点数。

代码

```
import sys

sys.setrecursionlimit(10**6)

def tarjan_scc(graph):
    n = len(graph)
    dfn = [-1] * n
    low = [-1] * n
    timer = 0
```

```

stack = []
on_stack = [False] * n
sccs = []

def dfs(u):
    nonlocal timer
    dfn[u] = low[u] = timer
    timer += 1
    stack.append(u)
    on_stack[u] = True

    for v in graph[u]:
        if dfn[v] == -1:
            dfs(v)
            low[u] = min(low[u], low[v])
        elif on_stack[v]:
            low[u] = min(low[u], dfn[v])

    if dfn[u] == low[u]:
        tmp = []
        while stack:
            top = stack.pop()
            on_stack[top] = False
            tmp.append(top)
            if top == u: break
        sccs.append(tmp)

for i in range(1, n):
    if dfn[i] == -1: dfs(i)
return sccs

```

```

def process_dag(n, sccs, graph):

    scc_id = [-1] * n
    for i, scc in enumerate(sccs):
        for u in scc:
            scc_id[u] = i

    num_scc = len(sccs)
    out_degree = [0 for _ in range(num_scc)]
    dag = [set() for _ in range(num_scc)]
    for u, neighbors in enumerate(graph):
        for v in neighbors:
            scc_u = scc_id[u]
            scc_v = scc_id[v]
            if scc_u != scc_v and scc_v not in dag[scc_u]:

```

```

        dag[scc_u].add(scc_v)
        out_degree[scc_u] += 1

    zero_out = []
    for i in range(num_scc):
        if out_degree[i] == 0:
            zero_out.append(i)
    if len(zero_out) > 1:
        return 0
    return len(sccs[zero_out[0]])

def main():
    n, m = map(int, input().split())
    graph = [[] for _ in range(n + 1)]
    for _ in range(m):
        a, b = map(int, input().split())
        graph[a].append(b)

    sccs = tarjan_scc(graph)

    print(process_dag(n + 1, sccs, graph))

if __name__ == '__main__':
    main()

```

(至少包含有"Accepted")

#52530551提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: **Accepted**

源代码

```

import sys

sys.setrecursionlimit(10**6)

def tarjan_scc(graph):
    n = len(graph)
    dfn = [-1] * n
    low = [-1] * n
    timer = 0
    stack = []
    on_stack = [False] * n
    sccs = []

    def dfs(u):
        nonlocal timer

```

基本信息

: 52530551
 题目: 02186
 提交人: 25n2500011619(xq)
 内存: 20668kB
 时间: 115ms
 语言: Python3
 提交时间: 2026-05-13 16:58:24

T01236: Network of Schools 238

SCC, <http://cs101.openjudge.cn/practice/01236/>

思路:

强连通分量+缩点+拓扑DP

分析

Tarjan 缩点将原图转化成DAG，统计每个强连通分量的出度入度，起点数量为 src ，终点数量为 des 。对于一个强连通分量，其中只要有一所学校获得新软件那么整个分量都能获得。

问题一

只要把软件给所有起点即可，答案为起点个数 src 。

问题二

结论

若 $num_scc = 1$ (只有一个强连通分量),则不需要连新的边，答案为 0。

若 $num_scc > 1$ ，则答案为 $\max(src, des)$ 。

证明

设缩点后的 DAG 中，起点（入度为 0）的集合为 P，终点（出度为 0）的集合为 Q。分以下两种情况讨论：

1. $|P| \leq |Q|$

① 若 $|P| = 1$ ，则只有一个起点，并且这个起点能走到所有点，只要将每一个终点都向这个起点连一条边，那么对于图中任意一点，都可以到达所有点，新加的边数为 $|Q|$ 。

② 若 $|P| \geq 2$ ，则 $|Q| \geq |P| \geq 2$ ，此时至少存在 2 个起点 p_1, p_2 ，2 个终点 q_1, q_2 ，满足 p_1 能走到 q_1 ， p_2 能走到 q_2 。（反证法：如果不存在两个起点能走到不同的终点，则所有的起点一定只能走到同一个终点，而终点至少有两个，发生矛盾，假设不成立）

那么我们可以从 q_1 向 p_2 新连一条边，那么此时起点和终点的个数都会减少一个（ p_2 不再是起点， q_1 不再是终点），因此只要以这种方式，连接新边 $|P| - 1$ 条，则 $|P'| = 1$ ，而 $|Q'| = |Q| - (|P| - 1)$ ，由 ① 得，当 $|P'| = 1$ 时，需要再连 $|Q'|$ 条新边，那么总添加的新边数量为 $|P| - 1 + |Q| - (|P| - 1) = |Q|$ 。

2. $|Q| \leq |P|$

与情况 1 对称，此时答案为 $|P|$ 。

综上所述， $num_scc > 1$ 时，问题二的答案为 $\max(|P|, |Q|)$ 即 $\max(src, des)$ 。

参考 luogu 题解 P2746 【USACO5.3 校园网Network of Schools】 - 洛谷专栏

<https://www.luogu.com.cn/article/4hmcb1h5>

代码

```

import sys

sys.setrecursionlimit(10**6)

def tarjan_scc(graph):
    n = len(graph)
    dfn = [-1] * n
    low = [-1] * n
    timer = 0
    stack = []
    on_stack = [False] * n
    sccs = []

    def dfs(u):
        nonlocal timer
        dfn[u] = low[u] = timer
        timer += 1
        stack.append(u)
        on_stack[u] = True

        for v in graph[u]:
            if dfn[v] == -1:
                dfs(v)
                low[u] = min(low[u], low[v])
            elif on_stack[v]:
                low[u] = min(low[u], dfn[v])

        if dfn[u] == low[u]:
            tmp = []
            while stack:
                top = stack.pop()
                on_stack[top] = False
                tmp.append(top)
                if top == u: break
            sccs.append(tmp)

    for i in range(1, n):
        if dfn[i] == -1: dfs(i)
    return sccs

def process_dag(n, sccs, graph):
    scc_id = [-1] * n
    for i, scc in enumerate(sccs):
        for u in scc:
            scc_id[u] = i

```

```

num_scc = len(sccs)
in_degree = [0 for _ in range(num_scc)]
out_degree = [0 for _ in range(num_scc)]
dag = [set() for _ in range(num_scc)]
for u, neighbors in enumerate(graph):
    for v in neighbors:
        scc_u = scc_id[u]
        scc_v = scc_id[v]
        if scc_u != scc_v and scc_v not in dag[scc_u]:
            dag[scc_u].add(scc_v)
            out_degree[scc_u] += 1
            in_degree[scc_v] += 1

zero_out = 0
zero_in = 0
for i in range(num_scc):
    if out_degree[i] == 0:
        zero_out += 1
    if in_degree[i] == 0:
        zero_in += 1
print(zero_in)
print(max(zero_in, zero_out) if num_scc > 1 else 0)

def main():
    n = int(input())
    graph = [[] for _ in range(n + 1)]
    for i in range(1, n + 1):
        neighbors = list(map(int, input().split()))
        for j in neighbors[:-1]:
            graph[i].append(j)

    sccs = tarjan_scc(graph)

    process_dag(n + 1, sccs, graph)

if __name__ == '__main__':
    main()

```

(至少包含有"Accepted")

#52581284提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: Accepted

源代码

```
import sys

sys.setrecursionlimit(10**6)

def tarjan_scc(graph):
    n = len(graph)
    dfn = [-1] * n
    low = [-1] * n
    timer = 0
    stack = []
    on_stack = [False] * n
    sccs = []

    def dfs(u):
        nonlocal timer
        dfn[u] = low[u] = timer
        timer += 1
        for v in graph[u]:
            if dfn[v] == -1:
                dfs(v)
            elif on_stack[v]:
                low[u] = min(low[u], dfn[v])
        if dfn[u] == low[u]:
            sccs.append(stack)
            stack = []
            on_stack[u] = False

    for u in range(n):
        if dfn[u] == -1:
            dfs(u)

    return sccs
```

基本信息

#: 52581284
题目: 01236
提交人: 25n2500011619(xq)
内存: 3800kB
时间: 21ms
语言: Python3
提交时间: 2026-05-19 11:41:45

2. 学习总结和个人收获

如果发现作业题目相对简单，有否寻找额外的练习题目，如“数算2026spring每日选做”、LeetCode、Codeforces、洛谷等网站上的题目。

本次考试AK，第一题上来没仔细看题，没判断同生日的人数大于1，导致答案错误，Debug 很久无果，只能放弃先去做二三两题，回来后才发现这个错误。

在考前花了一些时间练习SCC的题目，在考场上也是比较顺利的把 Tarjan 模板写出来然后提交AC，Network of Schools 这道题的答案很简单，但是证明比较麻烦，特别是证明存在这样的连边方式，要考虑入度和出度都为 0 的孤立点。（我在模考前一天晚上思考了很久还是晕晕的，后来参考洛谷题解才完全弄明白）

虫子的生活这道题被空间限制狠狠地制裁了，考场上最后是用 $N \times N$ 邻接矩阵 + DFS 染色的方法过的。考后学习了老师提供的复用对象的方式，同时练习了用 $2N$ 大小的扩展并查集，通过把异性的信息转化成同性信息（与自己的异性同类），就可以高效处理是否存在奇环的情况。